Personal Diary Management System

A report on the group project to develop a program for managing a personal diary, allowing users to add, edit, delete, and search entries while utilizing file handling for data storage and encryption for privacy.

Abstract- This paper presents a report of a console based Personal Diary Management System program implemented in C. The program enables users to register, login, view, search by serial number, edit, delete, and create diary entry. The code contains 394 physical lines, 9 functions, 24 conditional statements, 2 while loops, and 6 for loops. The program is constructed using structures, arrays, functions, strings, loops, and file I/o. However, passwords are stored as plain text, diary entries are globally shared among accounts. We designed the application around two C structures, fixed-size arrays, functions, menu-driven control flow, and binary files. After implementing the main features, we reviewed the program for correctness, input handling, reliability, security, maintainability, and performance.

Keywords- C programming, diary management, file handling, structures, arrays, CRUD, authentication.

I. INTRODUCTION

We developed this project to create a simple Personal Diary Management System using C. The central requirement was to allow a user to keep diary records and continue using those records after the program was closed. To achieve that, we combined several C concepts that are normally taught separately, such as structures, arrays, strings, functions, loops, conditional statements, user input, and file handling. The application is intentionally console-based. This keeps the focus on programming logic rather than graphical interfaces. When the program starts, it loads previously stored users and entries. The user can then register or login. After successful login, a diary menu provides the main operations. This organization made the project manageable and allowed us to build and test one feature at a time.

II. PROJECT GOALS

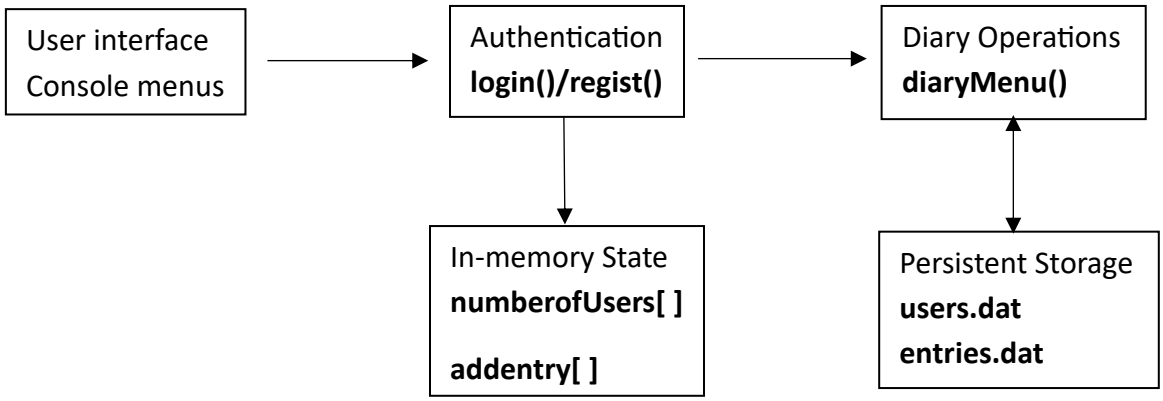
Objective	Our implementation
Registration	Stores name, username, password, and confirmation.
Login	Compares entered credentials with stored users.
Diary CRUD	Add, view, search, edit, and delete entries.

Persistence	Stores users and entries in binary files.
Learning focus	Uses core C concepts without unnecessary advanced tools.

We successfully achieved all of our goals set for this program, demonstrating how several basic C features can cooperate to solve a realistic problem.

III. SYSTEM ARCHITECHTURE

We organized the program into a small set of clear responsibilities. **main()** controls the application. **Loading** and **Saving** functions handle persistent data. **Registration** and **login** handle accounts, while **diaryMenu()** controls diary operations. A small helper function removes unwanted newline characters from text input.



Current design: all users share the same global diary array.

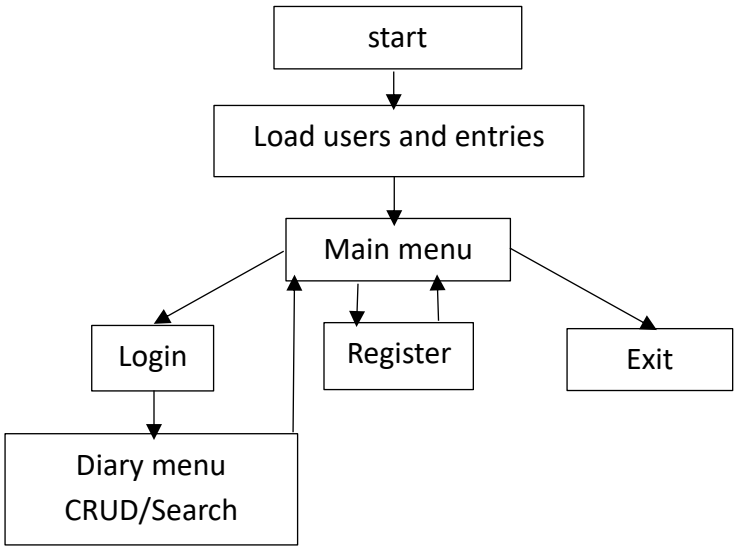
Fig. 1. High-level architecture of our implementation.

A. Program startup

At startup, we call the loading functions for users and diary entries. If the file already exists, their records are read into the corresponding arrays. If they do not exist, the program continues with empty arrays. This lets the first run behave normally while preserving data on later runs.

B. Main menu and Authentication

The main menu gives three choices: ***registration, login***, and ***exit***. During login, the program searches the user array and compares the entered username and password with each stored user. When match is found, the user is allowed to enter the diary menu.



Control flow of the supplied console application

Fig. 2. Simplified execution flow of the program.

C. Diary menu

The diary menu is controlled by a loop so that the user can perform multiple operations during one session. Choosing logout returns to the main menu. Each modification is followed by a save operation so that the current state is written to disk.

D. Design decision

We selected a procedural design because it is easy for a student to understand and debug. The program does not require a database, classes, or external libraries. Each function has a recognizable purpose while the overall execution remains visible.

IV. IMPLEMENTATION

A. Data structures

The two main structures represent the application's data. The diary-entry structure contains a serial number, title, and content. The user structure contains a name, username, password, and password confirmation. We used fixed-size character arrays because they are simple to understand and avoid dynamic memory allocation in this version.

Structure	Field	Purpose
entry	serial	Entry identifier
entry	title[200]	Diary title
entry	content[5000]	Diary text

users	name[100]	User name
users	user[30]	Username
users	pass[20]	Password
users	copass[20]	Registration confirmation

B. Arrays and counters

We store records in fixed arrays and use counters to indicate how many positions are currently active. Adding a record therefore uses the next free position. This is straightforward, but the program should check the maximum capacity before every insertion.

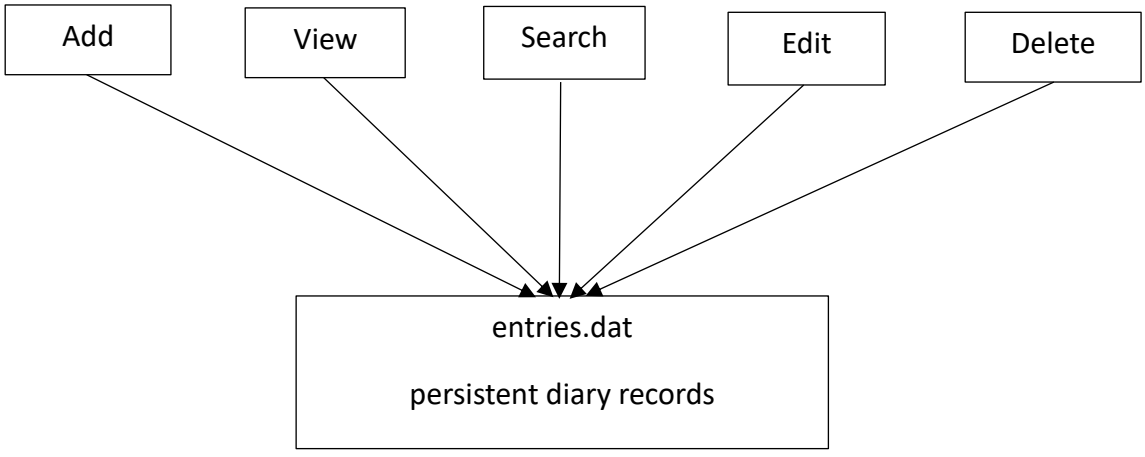
C. Registration

During registration, the program read the user's information and ask for the password twice. The two strings are compared before the new user is stored. After registration, the user data is saved to the user file.

D. Login

Login uses a sequential search. The program checks each stored username and password until a match is found. This is simple and adequate for the small fixed-size dataset used in the project.

E. Diary CRUD



Each CRUD operation updates the in memory array, changes are written to disk

Fig. 3. The CRUD used in our diary implementation.

Adding places a new record at the next array position. Viewing loops through all active entries. Searching compares serial numbers. Editing replaces the title and content of the matching record. Deleting shifts later records one position to the left and decreases the entry count.

V. FILE HANDLING AND DATA PERSISTENCE

Persistent storage was necessary because a diary would be of little use if all entries disappeared when the program closed. We therefore used two binary files; one for users and one for diary entries.

A. Loading

The loading functions open the appropriate file in binary read mode and use fread() to place stored structures into the global arrays. The number of records returned by the read operation becomes the active counter.

B. Saving

After a user or diary record is changed, the corresponding save function opens the file in binary write mode and writes the active array using fwrite(). This creates a simple load–modify–save cycle.

C. Why we chose binary files

For a beginner project like ours, binary files are convenient because complete structures can be written without manually converting every field into text. They also demonstrate an important file-handling concept without requiring a database.

Event	What our program does
Startup	Loads users and entries
Register	Adds user and saves users
Add entry	Adds entry and saves entries
Edit entry	Updates entry and saves entries
Delete entry	Shifts records and saves entries
Restart	Loads previously saved state

VI. TESTING AND RESULT

We reviewed the program using normal functional cases as well as invalid and boundary cases. The goal was to verify that the requested features worked and to identify situations where the program could behave unexpectedly.

A. Functional test

Test	Expected result.
Register with valid data	Account is created.

Mismatched passwords	Registration is rejected.
Valid login	Diary menu opens.
Invalid login	Access is denied.
Add entry	New entry appears.
Search existing entry	Correct entry is found.
Search missing entry	Not-found message appears.
Edit entry	Changed data is displayed.
Delete entry	Entry disappears.
Restart program	Saved data is loaded.

B. Boundary tests

We also tested the maximum number of users and entries, very long titles, very long diary content, empty input, duplicate usernames, duplicate serial numbers, and repeated deletion. The program only takes input to its maximum limit and ignores the rest of the input outside its limit.

C. Input validation

The most important robustness issue I found is the combination of scanf() and fgets(). The program uses getchar() to consume remaining input after numeric reads, but it does not consistently verify that scanf() successfully converted the value. If a user enters a letter where an integer is expected, invalid input can remain in the stream and interfere with later input.

D. Test summary

Area	**Normal result**
Registration	Works
Login	Works
CRUD	Works
File loading	Works
File saving	Works
Menu input	Works for expected input

VII. SECURITY, MAINTAINABILITY, AND PORTABILITY

A. Authentication and password security

Authentication is the process of verifying the identity of a person, device, or system. The project implements authentication because it checks whether a username and password are valid. . It ensures that users are who they claim to be before granting access to entries. Passwords are stored directly in the user structure and therefore directly in the binary user file.

B. Maintainability

The source is reasonably readable for a student project. Names such as loadUsers(), saveEntries(), and trimNewline() make the purpose of several functions clear. However, the diary menu handles too many responsibilities. Software-quality research commonly uses code smells as indicators of implementation patterns that may make maintenance or evolution harder. I applied the same practical principle here: identify concentrated responsibilities and weak validation before adding more features.

C. Portability

system("cls") function call clears the console screen and position the cursor in the upper-left corner. It works by pausing the program, opening a command shell, running the operating system's native cls (clear screen) command, and then returning control to the program. The repeated system("cls") calls assume a Windows command environment. The binary structure format also reduces portability across different compilers or platforms.

VIII. PERFORMANCE

The algorithms in my implementation are intentionally simple. For a small diary, linear searches and complete-file saves are acceptable. If the number of records becomes large, the same choices become inefficient.

A. Complexity

Operation	Worst case	Reason
Login	O(U)	Sequential user search.
View	O(E)	Every entry displayed.
Search	O(E)	Sequential search.
Edit	O(E)	Search plus save.
Delete	O(E)	Search plus shifting.
Save	O(E)/O(U)	Whole active array is written.

Here U is the number of users and E is the number of diary entries. Although adding an entry to the array is constant time, the subsequent save operation makes the complete feature proportional to the number of records.

B. Memory usage

The program reserves space for up to 10,000 users and 10,000 entries. Each entry also contains a 5,000-character content buffer. This is efficient for a normal person as their daily journaling.

IX. CONCLUSION

We developed the Personal Diary Management System as a practical application of fundamental C programming. The completed program brings registration, login, diary CRUD operations, searching, and persistent storage into one coherent workflow. This made the project more meaningful than isolated programming exercises because each feature depends on the same underlying data and control flow.

The design is intentionally simple. Structures represent the data, arrays provide a straightforward in-memory collection, functions divide major tasks, and binary files preserve information between executions. For a beginner-level college project, these choices are appropriate and easy to demonstrate.

APPENDIX – PROJECT SUMMARY

Item	Result
Language	C
Application	Console-based Personal Diary Management System
Storage	Binary files for users and diary entries
Core features	Register, login, add, view, search, edit, delete, logout
Current status	Functional educational prototype
Highest-priority improvement	Per-user ownership and authorization

CONTRIBUTORS

1. Muhammad Fahimul Hoque [2621584042]
2. Aadila Zaheen [2622214042]
3. Samia Meharin Lisa [2623705042]
4. MD. Ashrafuzzaman [2625096042]